

THERE IS SUCH THING AS A TAME BEAR

A Technical Reframing of AI Liability and the Applicability of the Wild-Animal Rule

Benjamin Gardner

Response to Trent S. Kannegieter, *Nondeterministic Torts: A Technical Approach to AI Liability*, 135 Yale L.J. 2719

(2026)

Abstract

In *Nondeterministic Torts: A Technical Approach to AI Liability*, Trent S. Kannegieter argues that because today's in-production large language models (LLMs) are unavoidably nondeterministic, the harms they cause should sound in strict liability under *ferae naturae*, the wild-animal rule that applies to bears and other untameable animals. The Note's methodological commitment—that liability doctrine must begin from how the technology actually works—is a heads-up contribution to the AI-liability conversation. This Response argues, however, that the Note's doctrinal potential is undermined by its key technical premise. The engineering record, including sources the Note itself cites, establishes that LLMs are in fact fully deterministic. LLM-adjacent nondeterminism is a removable artifact introduced by non-LLM elements, not an inherent propensity of the model; this Response supplies a fine-grained formal proof of LLM determinism to give that point rigor. Because *ferae naturae* turns on an inherent propensity, the Note's regime loses its trigger once nondeterminism proves non-inherent. Keying liability to a removable property like nondeterminism allows technology to outpace doctrine. Holding to the Note's own methodological commitment, this Response identifies Lipschitz continuity as the appropriate replacement for determinism. This mathematical property appears novel to legal scholarship and cannot be outpaced by technological advance.

Introduction

In *Nondeterministic Torts: A Technical Approach to AI Liability*, Trent S. Kannegieter builds a strict-liability regime on the premise that “today’s in-production [large language models (LLMs)] are unavoidably nondeterministic.”¹ Equating nondeterministic LLM environments with wild animals, the Note urges strict liability for alleged harms that LLMs cause to users and third parties under *ferae naturae*, the wild-animal rule.² The Note compares LLMs and bears: “Both entities can be trained . . . [b]ut such training does not erase underlying dangerous propensities altogether.”³

Kannegieter’s is a valuable and impressively rigorous contribution: it takes seriously the proposition that liability doctrine should be built from the mechanics of the technology it governs. Taking up that same commitment and pressing it one level deeper, however, this Response reveals that the Note’s key premise is unsubstantiated—indeed, that several sources the Note cites for the premise contradict it—and that a closer analysis of the underlying technical assertions indicates that nondeterminism is removable, not unavoidable. Thus nondeterminism is not the “untameable” propensity *ferae naturae* requires.

This Response’s title states its thesis in miniature. A koala bear, for example, is called a bear because it appeared to European colonists to be one; it is not. Similarly, an LLM may be called nondeterministic because it appears⁴ to its users unpredictable; it is not. As a result of

¹ Kannegieter, *supra*, at 2742. The quoted formulation appears in the Note’s survey of industry “consensus,” *id.*, but the premise that in-production nondeterminism is *unavoidable* is not confined to that passage. The Note separately asserts that “precise determinism is not possible,” *id.* at 2741, that “[d]emonstrating that a system exhibits stochasticity proves that the system exhibits nondeterminism,” *id.* at 2740 n.105, and that “[e]ven if it proves to be possible to create a completely deterministic LLM, no in-production model today exhibits these characteristics,” *id.* at 2742 n.116.

² See *id.* at 2719, 2728.

³ *Id.* at 2746.

⁴ Sending the same message to, e.g., ChatGPT from two different accounts may yield two different answers. This may *appear* to be nondeterminism but, when done purposefully, is in fact the effect of a hidden, deterministic number called a “seed” that parameterizes (acts as additional input to) the model. User input is not the entirety of

misnaming, the truism that “[t]here is no such thing as a tame bear”⁵ is flawed: various non-ursid “bears” may in fact be tamed. The Note applies a similarly ungrounded misnaming of LLMs as did those who misnamed the koala bear, and this Response replaces that naming with a rigorous and corrected taxonomy.

Because this Response holds to Kannegieter’s methodological commitment, its doctrinal refutation is internal rather than external, and dovetails with Kannegieter’s hopes of making existing doctrine more robust rather than answering AI with bespoke solutions.⁶ It underscores his commitment by demonstrating that, if laws are made from a place of an incomplete fluency regarding what it is they are meant to regulate, liability could be skirted by design-around—the designing of products such that they expressly dodge doctrinal triggers. Fluency is the only way to regulate *ex ante*—the only way to hold on tightly throughout AI’s protean shifts—rather than *ex post*, where each doctrinal fix is obsolete the moment the technology adversarially compensates.

Part I of this Response shows that the Note’s key premise is contradicted by the engineering record of the Note’s own sources, and supplies informal and formal proofs of LLM determinism. Part II demonstrates that the residual nondeterminism observed in current production deployments is a property of how inputs are supplied to the LLM—a property that is demonstrably eliminable—rather than of the model itself. Part III explains why the Note’s *ferae naturae* framework cannot survive the prior corrections, and Part IV shows that a regime keyed to a toggleable property, as the Note’s is, would induce its own evasion. Finally, Part V proposes

“input.” When an LLM gets the same input, hidden input included, it acts deterministically—unless the system includes a common but fixable issue as discussed *infra* Part II.

⁵ *Vredenburg v. Behan*, 33 La. Ann. 627, 629 (1881) (“There is no such thing as a ‘tame [b]ear.’”); RESTATEMENT (SECOND) OF TORTS § 507 cmt. e (A.L.I. 1977); see Kannegieter, *supra* note 1, at 2744-46 (deploying the *ferae naturae* analogy and conditioning it on the persistence of dangerous propensities despite taming).

⁶ Kannegieter, *supra* note 1, at 2776 (“this Note hopes to add to the AI-governance toolkit without creating a new tool at all”).

Lipschitz continuity as the durable technical anchor for which the Note was reaching, and identifies the open questions a re-keyed regime must answer.

The doctrinal payoff distinguishes this Response from mere technical correction. Along the way, the Response supplies two tools missing from the literature. Its first contribution is a fine-grained LLM determinism proof, a tool missing from the technical literature as well—formal evidence that a deterministic configuration exists as a demonstrable alternative design. Its second contribution is the introduction of the concept of Lipschitz continuity, the precise nucleus of AI’s product untestability—a formalization of the property legal scholarship approximates with words like “opacity” and “unpredictability.”

I. The Premise, Its Sources, and Its Mathematics

A. The Note’s Load-Bearing Citations

The Note cites a short list of technical sources to support its claims that, in LLMs, “precise determinism is not possible” and that “there is consensus that today’s in-production LLMs are unavoidably nondeterministic.”⁷ Kannegieter deserves credit for the attempt—the Note itself faults prior analyses for ignoring “the inner workings of the machines at hand,” and it practices what it preaches by grounding its premise in technical sources.⁸ None of those sources, however, substantiates the Note’s key claim that LLMs are unavoidably nondeterministic, and several contradict it.⁹ For example, the Note relies on Cooper, Frankle, and De Sa, but they

⁷ *Id.* at 2741, 2742 nn.109-13.

⁸ *Id.* at 2768.

⁹ One of the sources states that “Also known as neural networks, connectionist AI models are non-deterministic, meaning every time they are run, even with the same input, the outputs will be different.” M.L. Cummings, *Prohibiting Generative AI in Any Form of Weapon Control*, 39TH CONF. ON NEURAL INFO. PROCESSING SYS. 1, 2 (2025), <https://openreview.net/pdf?id=uEY7kQsiZz> [<https://perma.cc/RE86-Z7MD>]. Cummings does not substantiate the Note’s key premise, though. Hers is a policy position paper, rather than an empirical paper seeking formal correctness. Read in context, it is not a substantiated claim that—literally interpreted—all neural networks are non-deterministic, or that no two outputs of the same input are ever the same. Such literal claims would contradict the *infra* Part I.A Cooper, Chann, and He, and are easily falsified.

explain that “A particular model . . . is deterministic—always producing the same output given the same input.”¹⁰ He & Thinking Machines Lab state of the forward pass—a LLM’s output production—that “the forward pass in an LLM is in fact ‘run-to-run deterministic.’”¹¹ Chann points out that the LLM model “davinci-instruct-beta [is] deterministic.”¹² The Note’s other citations for the premise—industry commentary, journalism, and marketing materials—are not the kind of technical sources that could substantiate a claim about the mathematical properties of LLMs.¹³

B. LLMs are Mathematical Objects

Indeed, before an LLM is represented in code, it exists as a mathematical object. In the same way that addition is an abstract concept prior to someone coding a calculator, LLMs are abstract objects prior to someone coding them up. Proving the concept of addition deterministic does not necessarily mean that the calculator is deterministic, but it does establish that any

¹⁰ A. Feder Cooper, Jonathan Frankle & Christopher De Sa, *Non-Determinism and the Lawlessness of Machine Learning Code*, 2022 SYMP. ON COMPUT. SCI. & L. 1, 2-3. Cooper, Frankle, and De Sa locate non-determinism in the training process, not the deployed, fixed-weight model: “however, as we have discussed above, the entire procedure that produces such a model is not deterministic.” *Id.* at 3. Production LLMs being fixed models, this makes the training process irrelevant to a production context. This also counters the Note’s assertion that “[d]emonstrating that a system exhibits stochasticity proves that the system exhibits nondeterminism” in the sense that the Note uses this to mean that *all* of the system *will always* exhibit nondeterminism when in fact it is a claim about a *part* of the system that may even be temporary. Kannegieter, *supra* note 1, at 2740 n.105. Cooper, Frankle, and De Sa’s two statements combined say a system can be built stochastically and still be deterministic.

¹¹ Horace He & Thinking Machines Lab, *Defeating Nondeterminism in LLM Inference*, THINKING MACHINES (Sept. 10, 2025), <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference> [<https://perma.cc/4G5G-HNZ8>]. He’s thesis, per his title, is that nondeterminism is defeatable—that is, not an inherent characteristic of LLMs. His post contains empirical proof that the nondeterminism found in current production LLM pipelines can be fixed.

¹² Sherman Chann, *Non-Determinism in GPT-4 Is Caused by Sparse MoE*, SIMPLE THOUGHTS (Aug. 5, 2023), <https://152334h.github.io/blog/non-determinism-in-gpt-4> [<https://perma.cc/WW92-PZGU>] (“davinci-instruct-beta . . . is deterministic”). The Note cites this post—see Kannegieter, *supra* note 1, at 2741 n.108—for the claim that LLMs remain nondeterministic due to “architectures, like mixture of experts.” Chann says nearly the opposite: he locates the nondeterminism in the API’s “Sparse MoE architecture *failing to enforce* per-sequence determinism”—an implementation failure, not a property of the architecture—and his quoted line that “all *other* models . . . experience some degree of non-determinism” (emphasis added) itself presupposes deterministic models exist.

¹³ Kannegieter, *supra* note 1, at 2741-42.

observed nondeterminism in the calculator comes from somewhere other than addition’s essential properties.

After an LLM is trained, it is simply a large function, like $f(x) = y$. That large function is made up of a number of small functions, where the output of function g is the input to function f —this process of building $f(g(x))$ is called composition. These small functions are all deterministic (in mathematics, single-valued), and composing deterministic functions together always yields a deterministic function.¹⁴ Consider the small functions of adding, multiplying, exponentiating, and comparing. All of these functions are deterministic, and no matter how many of them are combined, their composition is also deterministic. A walk-through breaking down LLM components into these small functions follows, establishing that each is deterministic; a condensed formal proof follows.

C. Informal Proof of LLM Determinism

Trace a message through the mathematics in a standard LLM model and only seven primitive operations appear. The entirety of LLM operations is: indexing, addition, subtraction, multiplication, division by a strictly positive quantity, exponentiation (including roots), and comparison. None of these operations introduce randomness, and combining them will also never give randomness, no matter the order of their combination.¹⁵ Embedding lookup is indexing: index i retrieves row i , always. Positional encoding is multiplication and addition by fixed, precomputed constants. Linear layers ($Wx + b$) are matrix multiplication: multiplication

¹⁴ A mathematical function is “[a] relation from A to B that contains one, and only one, ordered pair (a, b) for every element $a \in A$.” KENNETH H. ROSEN, DISCRETE MATHEMATICS AND ITS APPLICATIONS 147 (8th ed. 2019)—in other words, a rule that links (relates) each input a to exactly one output b. The Note can then be understood to say “an LLM is a long series of [rules that link an input to exactly one output].” Asserting that a long series of rules that link an input to exactly one output itself maps to more than one output—is nondeterministic—is a contradiction.

¹⁵ More precisely: each is a single-valued function; chaining any number of single-valued functions yields a composite function that is itself single-valued; therefore the entire forward pass, from word IDs to output, is one single-valued mathematical function.

and addition. Attention is a bilinear product scaled by a fixed constant and optionally masked—multiplication, addition, division, and comparison. Softmax is comparison, subtraction, exponentiation, addition, and division. Normalization is elementary statistics—addition, multiplication, division, and a square root. Pointwise nonlinearities (ReLU, GELU) are comparison, multiplication, addition, and exponentiation. Finally, output selection is an argmax (a sequence of comparisons) at temperature zero or, at nonzero temperature, softmax plus a pseudo-random generator that is itself a deterministic¹⁶ function of its seed.¹⁷ All components of an LLM have been listed, and have been decomposed into primitive operations. No component can be said to introduce nondeterminism.

D. Formal Proof of LLM Determinism

After a thorough search, the author believes this Response's to be the first fine-grained formal proof of LLM inference determinism,¹⁸ making granular what computer science often

¹⁶ 2 DONALD E. KNUTH, *THE ART OF COMPUTER PROGRAMMING: SEMINUMERICAL ALGORITHMS* 3 (3d ed. 1997) (“There is a fairly obvious objection to this technique: How can a sequence generated in such a way be random, since each number is completely determined by its predecessor? The answer is that the sequence isn’t random, but it appears to be.”). This also counters the Note’s attempts to use temperature as proof of LLM nondeterminism. The Note reads “Technically, temperature can never be 0” to mean “there are technical constraints to temperature ever truly being zero.” Kannegieter, *supra* note 1, at 2741 n.108. The full quote: “Technically, temperature can never be 0—[probabilities] can’t be divided by 0. In practice, when we set the temperature to 0, the model just picks the [word] with the largest [probability] without doing the . . . calculation.” Chip Huyen, *Generation Configurations: Temperature, Top-k, Top-p, and Test Time Compute*, CHIP HUYEN (Jan. 16, 2024), <https://huyenchip.com/2024/01/16/sampling.html> [<https://perma.cc/6GU9-DGCA>]. That is: at temperature zero, the code just performs a simpler operation. No technical constraint exists. “Probability” is here imprecisely but losslessly substituted for “logit.” This Response makes a number of other simplifying substitutions: “message” stands in for “request,” “word” for “token,” “(LLM) operation” for “kernel,” and “AI” for “machine learning.” None of these substitutions bears on the determinism analysis. The formal proof (Section I.D) retains standard technical vocabulary.

¹⁷ See generally Ashish Vaswani et al., *Attention Is All You Need*, 31st CONF. ON NEURAL INFO. PROCESSING SYS. 1 (2017), <https://arxiv.org/abs/1706.03762> [<https://perma.cc/4PP3-QNBR>] (tracing entire architecture’s components, including specific inspections of embedding lookups, positional encoding, linear layers, attention, and softmax); IAN GOODFELLOW, YOSHUA BENGIO & AARON COURVILLE, *DEEP LEARNING* 168-72 (2016) (describing feedforward networks as compositions of affine transformations and fixed nonlinearities, as distinguished by the ReLU function from strictly linear functions); Jimmy Lei Ba, Jamie Ryan Kiros & Geoffrey E. Hinton, *Layer Normalization* ARXIV 2 (July 21, 2016), <https://arxiv.org/abs/1607.06450> [<https://perma.cc/RXL8-Z2H5>] (showing layer normalization’s equation).

¹⁸ Likely because the definitions of Turing machines or “functions” could be gestured to with the same result. It is a basic fact of computability theory that a single deterministic Turing machine—the standard formal model of computation, and what all everyday computers are mathematically—cannot generate nondeterminism. At each step a

asserts informally.¹⁹ Its value is evidentiary rather than novel—it does not prove a previously unknown proposition, but rather puts a public debate—as evidenced by the confusion in the Note’s nontechnical sources—to rest. This proof is redundant with the informal proof and is relevant primarily to the technical reader who would like to rigorously verify that LLMs are deterministic. Readers may resume at Part II without any loss of continuity or comprehension regarding the remainder of this response.

Setup

Fix a finite vocabulary V and let $X::= \bigcup_{n \geq 0} V^n$ be the set (infinite mathematically, though finite on hardware²⁰) of token sequences producible. A trained language model is a parameter vector $\theta \in \mathbb{R}^d$ (the flattened concatenation of parameter tensors) that determines a map

$$f_{\theta}: X \rightarrow \Delta(V),$$

where $\Delta(V)::= \{p \in \mathbb{R}^{|V|}: p_i \geq 0, \sum_i p_i = 1\}$ is the probability simplex over V . For each

$x \in X$, the value $f_{\theta}(x)$ is the next-token distribution given context x .

Turing machine’s next move is fixed by its current state and the symbol under its head. The Church-Turing thesis holds that anything “effectively computable” can be computed by such a machine. An LLM is effectively computable, so it can be computed by a deterministic Turing machine and therefore *can* be implemented deterministically. See generally MICHAEL SIPSER, INTRODUCTION TO THE THEORY OF COMPUTATION (3d ed. 2013).

¹⁹ He, *supra* note 11 (“the forward pass of an LLM inference server (like vLLM) can also be claimed to be deterministic”).

²⁰ On hardware, an additional $n \leq N$, where N is the model’s context window, renders X finite, contradicting “unbounded” output in “[LLMs] are nondeterministic, producing an unbounded range of varied outputs from identical inputs.” Kannegieter, *supra* note 1, at 2719. See Nadav Timor et al., *Accelerating LLM Inference with Lossless Speculative Decoding Algorithms for Heterogeneous Vocabularies*, 42ND INT’L CONF. ON MACH. LEARNING (Jan. 31, 2025), <https://arxiv.org/html/2502.05202v1> [<https://perma.cc/Z8ZL-SCL6>] (“Vocabularies are finite sets of [words].”); *Models Overview*, CLAUDE PLATFORM DOCS, <https://platform.claude.com/docs/en/about-claude/models/overview> [<https://perma.cc/LAN7-UZKS>] (last visited July 6, 2026) (“Max output 128k [words]”); *Task Budgets*, CLAUDE PLATFORM DOCS, <https://platform.claude.com/docs/en/build-with-claude/task-budgets> [<https://perma.cc/4CQC-NBN2>] (last visited July 6, 2026) (“Use max_[words] as the absolute ceiling that prevents runaway generation.”).

Lemma *The forward pass is a finite composition $f_\theta :: = \text{softmax} \circ g_L \circ g_{L-1} \circ \dots \circ g_1$*

in which every g_ℓ is single-valued.

Proof. Every g_ℓ is built from only (i) affine maps $z \mapsto Wz + b$ with W, b fixed entries of θ

(embedding lookup is such a map once each token is identified with its one-hot vector, with the fixed positional encodings supplying b), (ii) data-dependent bilinear products (attention's scores

$QK^\top$ and value aggregation $\sum_i \alpha_i V_i$), (iii) fixed pointwise nonlinearities (ReLU/GELU), and (iv)

fixed vector-valued maps (softmax, normalization). Matrix multiplication is single-valued over

the relevant domain $\mathbb{R}$, both bilinear products and fixed pointwise non-linearities are

single-valued by definition, and both softmax and normalization are compositions of pointwise

exponentiation, addition, and division by $\mathbb{R}^+$. By composition of single-valued functions, each

g_ℓ is a single-valued function. ■

Proposition f_θ is deterministic. That is, for all $x \in X$ the output $f_\theta(x)$ is uniquely determined by (θ, x) .

Proof. A composition of single-valued functions is single-valued: since each g_ℓ is single-valued,

so is $g_L \circ g_{L-1} \circ \dots \circ g_1$ by induction on ℓ . Hence $f_\theta(x)$ is uniquely determined by (θ, x) ; that is,

f_θ is deterministic. ■

Corollary *Decoding $f_\theta(x)$ into tokens is deterministic given a seed r_0 .*

Proof. Let a decoding rule be a map D and define autoregressive generation by

$x_{t+1} = D(f_\theta(x_{1:t}))$. Then:

- *Case: Temperature²¹ is zero.* If $D = \operatorname{argmax}$ (greedy, with any trivial deterministic tie-break), the entire continuation $x_{1:t} \mapsto x_{1:T}$ is a deterministic function of the prompt for any $T \geq t$.
- *Case: Temperature is nonzero.* If a new rule map D' samples from $f_\theta(x_{1:t})$ using pseudo-random state r_t , then $x_{t+1} = D'(f_\theta(x_{1:t}), r_t)$ is deterministic given r_t where r evolves deterministically from the initial seed. The only entropy enters through the externally supplied seed r_0 , not through f_θ . By composition of a pseudo-random generator (single-valued by construction) and f_θ , we have that D' is deterministic given r_0 . ■

Caveat

Determinism is about whether “the function gives one or more than one answer to the same input,” and what is categorized as input does not change with changes in perspective. Determinism is a claim about the *function itself*, not about how it is *perceived* from some standpoint. Unpredictability (epistemic) is not nondeterminism (ontological).

Put plainly: the fact that an outside observer cannot predict a function’s output from his or her partial knowledge of its input does not mean the function “gives different outputs from the same input.” There are many programs with outputs that defy human prediction—say a

²¹ Temperature is simply *softmax* and multiplication, as addressed *supra* in the Lemma. Seed is arbitrarily considered an external entropy source to retain a proof over the domain (θ, x) : were it to be considered input, making the domain (θ, x, r_0) , decoding would be unconditionally deterministic and the corollary would be moot. This bears directly on a competing mischaracterization. Cf. Eric Martínez, *Traditional and Computational Canons*, 39 HARV. J.L. & TECH. 287, 309 (describing a temperature sweep as ranging “from fully deterministic (0) to maximally stochastic (2)”). This framing and the Note’s cannot both be right: the former casts temperature zero as the deterministic pole of a stochastic axis, while the Note asserts unavoidable nondeterminism even at temperature zero. Indeed, neither is correct: all temperatures are equally deterministic. See *supra* note 16.

hand-coded chess bot—but unpredictability to users does not make those programs nondeterministic.

II. Instantiation on Hardware Can Preserve Determinism

The above establishes that LLMs are deterministic as mathematical objects. However, stepping outside the bounds of mathematical certainty and representing abstract truth on fallible hardware can introduce many unknown elements, some of which may conceivably reintroduce nondeterminism.²² The Note suggests such a step does, pointing to the non-associativity of floating-point arithmetic,²³ temperature,²⁴ and unboundedness²⁵—the way that computers imperfectly represent numbers, randomness that coders purposefully inject into models, and the concern that the LLM could run forever respectively. The premise of these worries is sound; it is possible that coding an LLM may introduce nondeterminism. However the conclusion that these are *unavoidable* issues is not. Each can be shown to be avoidable.

The way in which numbers are added on a computer, floating-point²⁶ arithmetic, which the Note calls out as a source of potential nondeterminism, is deterministic.²⁷ What it is not is associative, as in the associative property that makes $(1 + 2) + 3 = 1 + (2 + 3)$. Since each

²² On a case-by-case basis. We know it does not necessarily do so because Turing machines are deterministic. *See supra* note 18.

²³ Kannegieter, *supra* note 1, at 2741 n.108 (“the inherent challenges and time delays involved with using large amounts of distributed hardware that make it incredibly hard to replicate the exact order of interactions”).

²⁴ *See supra* note 16 for a counter.

²⁵ Kannegieter, *supra* note 1, at 2739 (“Nondeterminism, the ability of a model to generate an unbounded range of different outputs even given the same inputs, is a key feature of LLMs available to consumers in production today.”), 2741 n.108. Notice that the Note’s lack of separation between environment and LLM is what forces it to claim that specifically in-production LLMs are nondeterministic. But the LLM is the exact same mathematical object—in most cases, the exact same file—in production and out of it. Production is not a meaningful line to draw when considering the model itself; it is simply meaningful in that it introduces other components that may be nondeterministic.

²⁶ So named for the way in which the decimal point “floats” around in scientific notation, e.g., to 6.6743×10^{-11} from the decimal representation 0.00000000066743.

²⁷ IEEE STANDARD FOR FLOATING-POINT ARITHMETIC, IEEE STD 754-2019 (2019) (requiring each operation to return the correctly rounded result of the exact mathematical operation under a specified, deterministic rounding mode).

computer operation rounds,²⁸ $(a + b) + c$ and $a + (b + c)$ can, unintuitively, differ.²⁹ In a distributed, multi-machine deployment many floating-point operations run at the same time—one big addition equation being calculated piece by piece across many computers. Because addition order matters on computers, if the addition order changes (caused by, e.g., one computer going faster than another), the result will vary from run to run: a simple addition equation will be nondeterministic.

Kannegieter correctly identifies that today’s production LLM *environments* have some nondeterminism-introducing difficulty,³⁰ and points to technical authors who address floating-point arithmetic, but misses where they demonstrate a solution. Recent engineering work isolates this floating-point nondeterminism in LLM-related contexts: during what’s known as an “atomic add.” As He observes, “Although concurrent atomic adds do make an [LLM operation] nondeterministic, atomic adds are not necessary for the vast majority of [operations]. In fact, in the typical forward pass of an LLM, there is usually not a single atomic add present.”³¹ In the vast majority of cases where nondeterminism exists in an LLM environment, then, the Note’s floating-point concurrency concern is not what introduces it.

The culprit lies elsewhere: in how the message is sent from user to model. In many production deployments a user’s message to an LLM is “batched,” or combined, with others’ messages. When the model sees a big batch and a small batch it behaves differently, as the definition of determinism allows: its input size has changed. The size of a batch usually varies with server load—how many people are trying to message the model at once. Concretely, a

²⁸ Numbers are rounded because computers cannot, for example, store all the infinite digits of pi without an infinite amount of memory. See David Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, 23 ACM COMPUTING SURVS. 5 (1991).

²⁹ That is, $\text{round}(\text{round}(a + b) + c)$ and $\text{round}(a + \text{round}(b + c))$ can differ.

³⁰ Kannegieter, *supra* note 1, at 2741 n.108.

³¹ He, *supra* note 11. The word “usually” is necessary in the same sense that it is necessary, for formal correctness, to say “They usually do not sell cars at the grocery store.”

message sent at 2 a.m. alongside a few others is combined into a smaller batched input than the same message sent at 2 p.m. under heavy load, which would be placed in a larger batch. This is an arbitrary choice developers make for the sake of speed. From the user’s vantage the batching is invisible and his or her “same message” yields different results across runs; but from the perspective of the LLM it is not the same message—a user’s message is only a part of the LLM’s input, which has changed in size. Combined, load-dependent inputs change the LLM input out from underneath the user message, which explains the user’s experienced nondeterminism.

This non-batch-invariance (batched operations not staying a constant size) is true nondeterminism,³² but it lives in the “sending” part between the user and the model, not in the model itself. It is nondeterminism *associated with* LLMs, not nondeterminism *caused by* LLMs. Make LLM operations not change with the size of input (batch invariance)—so that a message computes identically whether batched with two messages or two hundred—and the nondeterminism is removed. This is precisely the sense in which there is no such thing as a nondeterministic LLM—only a nondeterministic message-serving environment with a deterministic LLM inside it. Nondeterminism is introduced between when the user sends a message and the LLM receives it. This only makes it *look* like “production LLMs” are nondeterministic.

Removal of this form of nondeterminism has been demonstrated, not merely theorized. Implementing the batch-invariant operations as mentioned, researchers demonstrated how a nondeterministic LLM environment was changed into a deterministic one. Using a production server in the two batch variance configurations, He sent the same message to a production-sized model a thousand times, reducing from eighty different responses (constituting nondeterminism)

³² Only the size confounds; each user’s messages are walled off from the others by an internal mechanism.

to a single, identical response (constituting determinism).³³ This experiment is empirical proof of an LLM environment being shown to be deterministic across the whole pipeline: throughout the whole system, user message to LLM output. This is wider than the scope of determinism already proven: within the LLM itself, combined LLM input to LLM output. The cost of this change—with batch-invariant operations in place, the model ran more slowly—does not present an insurmountable obstacle.³⁴ Determinism is thus a specific, nameable, and optional infrastructure choice *even in production configurations*. This counters all objections that floating-point non-associativity, architecture, or any other standard feature makes LLMs nondeterministic.

This has legal implications: for purposes of doctrine, deterministic LLMs are available, demonstrated, and reasonable. Similarly, the Note’s prediction that firms will *select* nondeterminism even if determinism becomes available³⁵ concedes the very point that dismantles *ferae naturae*: determinism is a choice.³⁶ An in-production LLM is tameable.

III. *Ferae Naturae* Keyed to Nondeterminism Is Shaky Ground

The technical correction reshapes, but does not dissolve, the Note’s liability framework. In fact, the doctrinal reversal demanded by the updated technical facts only underscores the importance of the Note’s suggested approach of a close technical-legal bridge. The new technical

³³ The runs were conducted at temperature zero. He, *supra* note 11.

³⁴ The variable batch size existed to serve high message volume quickly; the speed cost of batch invariance relative to a leading alternative was roughly sixty percent. *Id.* (“performance is not disastrous”).

³⁵ See Kannegieter, *supra* note 1, at 2741.

³⁶ *But see id.* at 2742 n.116 (acknowledging He, *supra* note 11, yet discounting it as not in production and predicting that firms will “still select nondeterministic options”). The demonstration runs on vLLM—a production inference server—with Qwen3-235B, a production-scale model; it was production-grade even if not deployed to outside users, and could trivially be deployed. The prediction that firms will select nondeterminism concedes that determinism is a choice, which is part of the predicate this Response presses.

understanding that LLMs are deterministic does, however, counter the strict-liability vehicle the Note prefers.

Strict liability for wild animals rides on a particular fact: the dangerous propensity is “characteristic of wild animals of the particular class”³⁷ and persists despite the keeper’s best efforts to tame or confine it. Kannegieter chose the analogy well: *ferae naturae* is precisely the doctrine one would want if the premise held, because its core—inherence despite the keeper’s best efforts—maps exactly onto the Note’s account of nondeterminism. The keeper of a bear is liable even after taking every precaution, because the bear’s nature is fixed and beyond the keeper’s control. That inherence is the analogy’s engine. But the nondeterminism of LLMs’ production environments today is not inherent. “There is no such thing as a tame bear”; yet there *is* such a thing as a deterministic LLM. Nondeterminism is not “characteristic of . . . the particular class.”

IV. Liability Keyed to a Togglable Property Fails

Keying strict liability to nondeterminism is a mistake because the nondeterminism trigger is not a technical invariant—instead, it is changeable, and orthogonal to harm. A doctrine built on a togglable property risks easy evasion.

Consider how the Note’s own tort regime would fare if courts adopted it. Assume a hospital network deploys an agentic, LLM-based clinical tool that renews prescriptions without a physician in the loop. The tool errs; a patient, A, is harmed. Because the LLM environment is nondeterministic, the court holds the deployer strictly liable under *ferae naturae*—the

³⁷ RESTATEMENT (SECOND) OF TORTS § 507(2) (A.L.I. 1977), *quoted in* Kannegieter, *supra* note 1, at 2744.

untameable variance is the dangerous propensity, and no precaution excuses it. The legal rule based on nondeterminism is thus announced.

Announcing a rule that nondeterminism triggers liability figuratively puts a bounty (a dollar amount based on the size of the expected damages) on the nondeterminism in the system. Part II showed its elimination is a modest, one-time engineering step. The developer collects the bounty: he enables batch-invariant inference as described in He. The change is motivated by escaping liability, not harm reduction—and harm is not reduced.³⁸ The doctrine's first and only observable effect was to induce the disappearance of its own trigger. Each future plaintiff starting with patient B cannot point to nondeterminism and instead reaches for a strengthened (given the deterministic alternative design) design defect claim. The wild animal rule has become irrelevant.

Keyed to a mutable implementation property, a doctrine's coverage expires with the property. Applied instead to immutable³⁹ mathematical characteristics and harm, it can survive any speed of technical progress. The measure of a successful technical anchor is whether it would still describe the technology after any number of adversarial attempts by those regulated to achieve design-around.

V. Lipschitz Continuity as a Durable Technical Anchor

A. From Nondeterminism to Lipschitz Continuity

³⁸ Unless it is reduced incidentally. See *infra* Part V for an explanation of why discontinuity is the relevant hazard, not run-to-run variance.

³⁹ Characteristics immutable or untoggable in the sense that they cannot be eliminated without eliminating the danger.

The present task, then, is to specify the precise and immutable property that makes an LLM’s hazards⁴⁰ foreseeable despite its scale. Current legal scholarship, in its attempts, uses vague terms: opacity, unpredictability, breakdown of “linear causation” and product-testing assumptions. Nondeterminism was a serious candidate, selected through exactly the right kind of rigorous mechanistic reasoning; it was simply the wrong invariant.⁴¹ All were approximations of the relevant mathematical invariant: Lipschitz continuity. Lipschitz continuity is the only⁴² property that lets finite testing certify astronomically large probability spaces by interpolation—interpolation meaning, e.g., “if the stove is at 300°F when the dial is at 3 and 350°F at 4, one can reasonably assume setting the dial in the middle will be between those two temperatures”⁴³—and interpolation is the only route left once exhaustive verification and distribution-level statistical guarantees fail, as they often do at LLM scale. Despite being the exact descriptor for product safety tests’ failure to handle LLMs, this term appears novel in legal scholarship.⁴⁴

⁴⁰ A problem of LLM hazard is that, hypothetically, the model can produce any combination of words in its vocabulary. *See supra* Part I.D for the “set of [word] sequences producible.” However, for safety, the model must be constrained to eliminate large swaths of those combinations, e.g., sentences that constitute hate speech. Since it is impossible to type out every disallowed sentence into a blocklist, the solution can be thought of mathematically: turning all sentences into a point on a (multidimensional) graph, similar sentences next to each other, and making certain areas of that graph unreachable by any input. However, no matter how many inputs are tested, it is impossible to confirm that a particular disallowed area is in fact unreachable without additional information. A user not having hit a disallowed area *yet* isn’t proof a user *can’t* hit it.

⁴¹ *See* Kannegieter, *supra* note 1, at 2768 (nondeterminism means the complete set of eventual outputs cannot be examined). The ability to examine all possible outputs is a separate problem from which one of those possible outputs will occur during a particular run. A disconfirming example—nondeterminism that can be fully examined—is a bag with one white marble and one black marble being pulled out. The output is either white and then black or black and then white, nondeterministically. Now imagine the Note’s argument is that it is size and nondeterminism together that make an output unexaminable. A deck of 52 cards shuffled at random is both large and nondeterministic, and its outputs remain completely examinable (that is, characterizable). Some principle other than nondeterminism must explain why the complete set of eventual LLM outputs cannot be considered examined.

⁴² If we include generalizations (other types) of Lipschitz continuity such as the modulus of continuity. Hölder continuity, of which Lipschitz continuity is a type, is too broad because it permits infinitely steep slopes. *See* LAWRENCE C. EVANS, *PARTIAL DIFFERENTIAL EQUATIONS* § 5.1 (2d ed. 2010) (defining Hölder spaces). For exponents $\alpha < 1$, the Hölder bound $C\|x - y\|^\alpha$ permits unbounded difference quotients—e.g., $f(x) = \sqrt{x}$ is Hölder continuous with $\alpha = 0.5$ yet has arbitrarily steep secant slopes near zero—whereas Lipschitz continuity (the $\alpha = 1$ case) bounds every slope by L .

⁴³ More rigorously, interpolation is “sampling a large input space and inferring that untested points between the samples behave with bounded similarity.”

⁴⁴ *See* Issar Birger, *Negligence as Optimization Puzzles: A New Theory of Negligence* (2015) (J.S.D. dissertation, Yale Law School),

<https://openyls.law.yale.edu/server/api/core/bitstreams/8f931ff0-166e-412d-8c2d-da0f6a95fdb1/content>

A function is Lipschitz-continuous when the change in its output is bounded by a constant multiple L of the change in its input: $\|f(x) - f(y)\| \leq L\|x - y\|$.⁴⁵ Conventional engineered systems approximate this behavior, which is what makes ordinary product testing tractable. Because nearby inputs yield nearby outputs, a tester can interpolate. This is not the case for LLMs.

The input and output probability spaces of an LLM are enormous, but this alone does not defeat tort analysis—large, partitionable spaces are nothing new. What is distinctive is that, in addition to being enormous, an LLM’s input-output map is not considered⁴⁶ Lipschitz-continuous, so inputs that are very similar may yield wildly distant outputs: setting the stove dial to 3.5 may yield 700°F.⁴⁷ The interpolation validity assumption underlying conventional product testing then fails, since sampled inputs cannot certify the untested interior.

[<https://perma.cc/WB2T-B3EG>]. As a closest match, Birger discusses continuity in a similar (negligence) context, but not Lipschitz continuity. Continuity is insufficient for product testing because, absent an L -bounded $\|x - y\|$, no finite sampling grid can guarantee that a point between x and y behaves boundedly similarly. *See also* Luke Minton, *Strict Algorithmic Scrutiny: A Formulation of Algorithmic Procedural Fairness Through Affirmative Action Jurisprudence* (2020) (A.B. thesis, Harvard College), <https://nrs.harvard.edu/URN-3:HUL.INSTREPOS:37364713> [<https://perma.cc/VVD9-PBTY>] (applying Lipschitz continuity to affirmative action—an applied-math paper whose application happens to be law, not a law paper).

⁴⁵ This is the formulation of Lipschitz continuity on a normed vector space (the model’s representation space).

⁴⁶ Technically LLMs *are* Lipschitz-continuous most of the time, however they are “considered” non-Lipschitz. This is because L (the bound constant) needs to be small to be useful. L bounds grow very quickly in LLMs, so they have a uselessly large L —so as a shorthand they are called non-Lipschitz when in reality a more correct term would be “not usefully Lipschitz.” This means that, were Lipschitz continuity encoded legally, a usefully-bounded L would need to be specified instead of strictly “non-Lipschitz.” This is a somewhat pedantic detail. Similarly pedantic, it is also technically possible to give LLMs a small L , but at substantial cost to model expressivity. *See* Hyunjik Kim, George Papamakarios & Andriy Mnih, *The Lipschitz Constant of Self-Attention*, 38TH INT’L CONF. ON MACH. LEARNING 1 (2021), <https://arxiv.org/abs/2006.04710> [<https://perma.cc/62UC-3S5U>] (proving that the standard dot-product self-attention used in transformer LLMs is not Lipschitz on an unbounded domain—that is, not usefully Lipschitz—and that achieving a Lipschitz guarantee requires substituting an alternative attention mechanism); *see also* Cem Anil, James Lucas & Roger Grosse, *Sorting Out Lipschitz Function Approximation*, 36TH INT’L CONF. ON MACH. LEARNING 1 (2019), <https://arxiv.org/abs/1811.05381> [<https://perma.cc/3P8M-SQ9L>] (demonstrating that enforcing Lipschitz constraints on standard neural architectures sacrifices expressive power absent specialized designs).

⁴⁷ Christian Szegedy et al., *Intriguing Properties of Neural Networks*, ARXIV (Feb. 19, 2014), <https://arxiv.org/pdf/1312.6199> [<https://perma.cc/39FJ-CHMB>] (“we find that deep neural networks learn input-output mappings that are fairly discontinuous to a significant extent. We can cause the network to misclassify an image by applying a certain hardly perceptible perturbation”). Objections based on the continuous nature of a dial as opposed to the discrete LLM input of words may be answered by pointing to LLMs’ continuous internal representations.

Sampling satisfies a duty to test only where interpolation between samples is valid; interpolation is valid only under a bounded L ; therefore, for non-Lipschitz products, a sampling-based testing record is not evidence of due care but evidence the developer used insufficient tests and deployed anyway. The reason Lipschitz continuity is not vulnerable to design-around in the same way nondeterminism was is because Lipschitz continuity is exactly the technical guarantee of LLM conformance with testing assumptions. Were an engineer to make a usable LLM Lipschitz-continuous, they would make an LLM that conforms to standard product testing and its safety guarantees, even in the absence of Note-suggested guardrails.

In the strongest reading of Kannegieter, wherein “nondeterminism” is a nontechnical shorthand for unpredictability over a vast prompt space, the Note’s logic itself points to Lipschitz continuity as the correct technical term for the logic already present. This Response’s view is that the strict liability the Note favors, re-keyed to Lipschitz continuity, survives the challenge of adversarial design-around.

B. Open Questions for a Re-Keyed Regime

Various questions challenge the Note’s doctrinal recommendations after the proposed replacement of nondeterminism with Lipschitz continuity. Such questions must be answered in future work to ensure the stability of the suggested regime.

Does a re-keyed *ferae naturae* survive assumed risk? Assumed risk is a standard defense to strict liability,⁴⁸ and it’s reasonable to expect that most LLM harms befall those who purposefully sought the model’s output. Does strict liability then reach only plaintiffs who never intentionally used an LLM-involved system at all—and if so, how much of the regime survives?

⁴⁸ RESTATEMENT (THIRD) OF TORTS: LIAB. FOR PHYSICAL & EMOTIONAL HARM § 24(a) (A.L.I. 2010) (“Strict liability . . . does not apply . . . if the person suffers physical or emotional harm as a result of making contact with or coming into proximity to the defendant’s animal or abnormally dangerous activity for the purpose of securing some benefit from that contact or proximity”).

Does the technical basis for discarding nondeterminism overhaul negligence in the same way it did strict liability? Deterministic batch-invariant LLM configurations provide, *prima facie*, a “reasonable alternative design” to nondeterministic configurations. There are, however, speed costs to implementing deterministic LLM environments, which would induce disparate effects regarding applications that rely on real-time reply speed.⁴⁹ Would it be helpful to define different groups of LLM use for which different burden calculations would apply?

Conclusion

Determinism is a toggle, not a nature. The bear cannot be tamed; the model can—fix a seed, run batch-invariant operations, and every LLM is deterministic. Such a change in technical reality shifts the legal ground substantially. The nondeterminism-related harm the Note fears, then, is not best addressed as the escape of something wild but as the foreseeable result of a design choice. This change preserves the Note’s mechanistic insight while correcting its mechanics and helping ensure a stable tort regime even as technology continues its advance. To guarantee AI can never wriggle out from underneath its regime, law should turn its attention from nondeterminism to a property that is future-proof.

These stakes are more than academic. On the other end of the incidents and programs that motivated the Note are patients whose prescriptions are renewed by a model,⁵⁰ companies that lose the entirety of their work product,⁵¹ and vulnerable users failed by automated mishandlings of mental health counseling.⁵² A liability regime that focuses on the wrong technical property fails all of them at once: it invites evasion by the firms it means to discipline and denies recovery to the plaintiffs it means to protect. Kannegieter’s great service is in insisting that tort law learn

⁴⁹ See He, *supra* note 11.

⁵⁰ Kannegieter, *supra* note 1, at 2725 & n.25.

⁵¹ *Id.* at 2721-22 (explaining the Replit incident).

⁵² *Id.* at 2770 n.240.

the mechanics of what it regulates. Taking that instruction seriously means anchoring doctrine to the property that will still describe the technology after every adversarial design-around has been tried—and that anchor is Lipschitz continuity.